

VANTAGE

Architected comprehensive specification mapping agents, files, and problem-solving mechanisms

Vantage — Complete Agent & Module Reference

This is the full technical breakdown: every "agent" (module), what problem it solves, what files it lives in, and how it works internally. Not all of these are literally AI agents — most are deterministic logic modules — and that distinction is called out explicitly for each one, since that's a core selling point of the project.

1. Simulation Agent

Problem solved: #3 — transaction executes differently than what the user was shown (bad price, unexpected revert).

Type: Deterministic (no AI)

What it does: Takes a prepared but unsigned transaction and runs it against the *current* live blockchain state using a read-only call, without spending any gas or committing anything. Compares the result to what the user was originally quoted.

Files:

lib/simulate.ts → core simulation logic, wraps viem's simulateContract/call

lib/riskScore.ts → computes % difference between quoted vs simulated output

app/api/simulate/route.ts → API endpoint exposing this to frontend + external callers

contracts/src/MockAMM.sol → the pool being simulated against

How it works internally:

1. Receives { from, to, data, value, quotedOutput }
 2. Calls viem.simulateContract() against current chain state
 3. Extracts actual output from simulation result
 4. Diffs quotedOutput vs simulatedOutput → percentage delta
 5. Also captures revert reason if the simulation fails entirely
 6. Returns structured JSON: { success, simulatedOutput, deltaPercent, revertReason, gasEstimate, riskLevel }
-

2. Conflict / Stale-State Detector

VANTAGE

Problem solved: #3 (specific case — transaction becomes invalid because someone else changed contract state first, e.g. claimed a slot, drained liquidity)

Type: Deterministic (no AI)

What it does: Same simulation mechanism as above, applied to a state-dependent contract (a claim/slot contract) to catch the simplest, cleanest version of "your transaction will fail because the world changed since you loaded the page."

Files:

contracts/src/MockClaim.sol -> single-slot claim contract for demo

lib/simulate.ts -> reused, same simulation call

script/ClaimRace.s.sol -> Foundry script simulating another wallet claiming first

How it works internally:

1. User's UI shows "1 slot available" (read from chain)
 2. Before the user signs, simulate.ts re-checks current state
 3. If another wallet has claimed it in between (via the demo script), the simulation returns a revert ("No slots left")
 4. Guardian blocks with: "This will fail — the slot was already claimed. Don't submit."
-

3. Transaction Watcher (Async-Mempool Agent)

Problem solved: #7 — transaction appears to vanish (null status) due to Monad's asynchronous mempool/propagation model

Type: Deterministic (no AI) — this is the Monad-specific differentiator module

What it does: After a transaction is submitted, continuously polls its status and disambiguates between states that all *look* the same to naive polling logic (null) but mean very different things.

Files:

lib/watcher.ts -> polling loop + status classification

app/api/watch/route.ts -> GET endpoint, returns current status for a txHash

How it works internally:

1. Receives a txHash

VANTAGE

2. Polls `viem.getTransactionReceipt()` on an interval
 3. Classifies the result into one of: `CONFIRMED`, `NULL_PENDING` (still alive, just async-delayed), `STUCK` (pending too long, likely nonce/gas issue), `DROPPED` (no longer found anywhere)
 4. Returns `{ status, secondsPending, recommendation }`
 5. This same endpoint is designed to be called by AI agents doing step-by-step confirmation logic, not just the human UI — solves the "bot gets stuck in a logic loop" problem too
-

4. Congestion Detector

Problem solved: #4 — execution slows down or behaves unpredictably during high contract activity

Type: Deterministic (no AI)

What it does: Checks how many recent calls have hit the target contract and flags it as "hot" if activity is abnormally high, warning the user that execution may be delayed or state may shift quickly.

Files:

`lib/congestion.ts` → call-count tracking logic

`app/api/congestion/route.ts` → endpoint returning current heat score for an address

`script/Congest.s.sol` → Foundry script firing real load at MockAMM for the demo

How it works internally:

1. Tracks recent transaction count to a given contract address over the last N blocks
 2. `Congest.s.sol` fires 15-20 real transactions at MockAMM right before the demo swap, so the "hot contract" state is genuine, not narrated
 3. Computes a heat score, returns `{ heatLevel, recentCallCount, recommendation }`
-

5. Approval Risk Checker

Problem solved: unlimited token approvals (a general wallet-safety issue, made worse by fast-moving DeFi activity where users approve quickly without reading)

Type: Deterministic (no AI)

VANTAGE

What it does: Decodes any transaction calldata that matches an approve(spender, amount) call and flags it if the amount is unlimited (type(uint256).max).

Files:

lib/approvalCheck.ts -> calldata decoding + threshold check

app/api/approval/route.ts -> endpoint for this specific check

How it works internally:

1. Decodes tx data field using the ERC-20 approve ABI
2. Compares amount against $2^{256} - 1$
3. If it matches (or is suspiciously large), flags HIGH risk and suggests the exact amount needed instead

6. AI Explainer (the only actual AI/LLM component)

Problem solved: translating dense numeric risk output into something a non-technical user understands — not a detection problem, a communication problem

Type: AI (LLM call) — explicitly scoped to explanation only, never computes risk

What it does: Takes the structured JSON output from modules 1-5 and generates one plain-English warning sentence.

Files:

lib/explain.ts -> builds the prompt, calls the LLM API

app/api/explain/route.ts -> endpoint wrapping this

How it works internally:

1. Receives structured risk data, e.g. { type: "swap", quotedOutput: "100 USDC", simulatedOutput: "88 USDC", deltaPercent: 12, riskLevel: "HIGH" }
2. Sends this as context to an LLM with a strict instruction: describe this result in plain English, do not add numbers or claims not present in the input
3. Returns a single sentence, e.g.: "This trade will give you 12% less than expected — you may want to wait or reduce your trade size."

7. Risk Log

VANTAGE

Problem solved: no problem directly — this is the "proof of value" layer for the pitch, not a detection mechanism

Type: Deterministic (no AI) — just storage

What it does: Records every simulated transaction, its risk score, and outcome, so the demo can end with a real number instead of a claim.

Files:

lib/db.ts -> SQLite wrapper (better-sqlite3 or Prisma)

prisma/schema.prisma -> (if using Prisma) table schema

Schema (conceptually):

```
RiskEvent {  
  
  id, walletAddress, txType, riskLevel,  
  
  deltaPercent, revertReason, timestamp, outcome  
  
}
```

How it works internally: every time modules 1-5 produce a result, it's written here. The frontend queries this table to show "Vantage has caught N risky transactions this session."

Summary Table

# Agent/Module	Problem Solved	AI Involved?	Core File
1 Simulation Agent	#3 (bad execution)	No	lib/simulate.ts
2 Conflict/Stale-State Detector	#3 (state race)	No	contracts/MockClaim.sol
3 Transaction Watcher	#7 (vanished tx)	No	lib/watcher.ts
4 Congestion Detector	#4 (slowdowns)	No	lib/congestion.ts
5 Approval Risk Checker	Unsafe approvals	No	lib/approvalCheck.ts
6 AI Explainer	Communication clarity	Yes	lib/explain.ts

VANTAGE

# Agent/Module	Problem Solved	AI Involved?	Core File
7 Risk Log	Proof/demo credibility	No	lib/db.ts

Key line for judges: "6 of these 7 components are pure deterministic code — real simulation, real math, real on-chain data. Exactly one uses AI, and its only job is turning already-computed numbers into a sentence a human can read."